P390

Britt Anderson

2025-01-01

Table of contents

Preface **1**

1 Introduction **3**

2 How can a computer help us in our research? **5**
2.1 Research Project Support . . . 5
2.2 Open Research Practices . . . 5

3 IDEs **7**
3.1 Required for this Course: VS Code . . . 7
3.2 For the Hardcore . . . 7

4 Using VSCode **9**

5 Jupyter Notebooks **11**

6 Terminals **13**
6.0.1 Commands to try (many have options) . . . 13
6.1 Additional Introductory Material . . . 14
6.2 Getting work done: scripting . . . 14
6.3 Talking to other computers in the terminal . . . 14
6.3.1 SSH . . . 14

7 Version Control **15**
7.1 Git . . . 15
7.2 An old overview of git and Github . . . 15
7.2.1 Getting Git . . . 15
7.2.2 Configuring Git . . . 15
7.3 Other version control system . . . 15
7.4 Git is not Github . . . 16
7.5 Using git with github . . . 16
7.5.1 Creating an ssh key with your terminal . . . 16
7.6 Understanding Git and the Workflow in Pictures . . . 16
7.7 Principal Terms Terms for Git . . . 16

8 Lab Notebooks **19**

9 Writing Scientific Documents **21**
9.1 IDEs and Writing up your Research 21
9.1.1 VSCode 21
9.1.2 Some helpful links to get you started 22
9.1.3 RStudio 22
9.2 Scientific Publishing Tools 22
9.3 Things You Can Do With Quarto 23
9.3.1 Making a Website with Quarto and Github Pages 23
9.3.2 Journal Article Authoring 26
9.3.3 Presentations 26

10 Coding **29**
10.0.1 Languages 29
10.0.2 Programming Languages 29

11 Programming Experiments **41**
11.1 All-in-one options 41
11.2 Python 42
11.2.1 Pygame 42
11.3 Javascript 44

12 In Lab Experiments **45**
12.0.1 Online Experiments 45

13 Handling Data **47**

14 Knowledge Management **53**
14.1 (Zettlekasten) 53
14.2 Reference Management 55
14.2.1 Finding Citations 55
14.2.2 More on Citation Management 55
14.2.3 The last thing you want to do … 56
14.3 Using References With Quatro and VS Code 56
14.4 Using References With Overleaf 57

15 Testing Your Javascript **59**
15.1 Run your own server locally 59
15.1.1 XAMPP 59
15.1.2 In Lab Experiments 59
15.1.3 Online Experiments 59

16 Large Language Models **61**
16.1 Run locally 61
16.2 How to? 61
16.2.1 Key Features of GPT4ALL 61

16.3 How-to . . . 62
16.4 Using LLMs for Dynamic Research . . . 62
16.5 Why Might You Want To Do This? . . . 63
16.6 Next steps . . . 63

17 Databases and management 65
17.0.1 Datascience . . . 65
17.0.2 Grant Funding . . . 65

18 Summary 67

Appendices 69

A Mac Related Instructions 69
A.1 Homebrew . . . 69
A.1.1 Texlive . . . 69
A.2 UV . . . 69
A.3 Quarto . . . 69

B Python 71

C Virtual Environments 73
C.1 Methods . . . 73
C.1.1 Creating and Activating the Venv . . . 73
C.1.2 The Terminal is All You Need . . . 74
C.2 Visual Studio Code . . . 74
C.2.1 Quarto Complications . . . 75

Preface

This course is an effort to expose research minded undergraduate students in psychology to a variety of computational and programming tools that they can use to ease their research tasks. It is also intended to provide them practice at using such tools. Although I hope the particular tools that I choose for this are useful now, they may not be in the future. Things change fast in this area: smartphones are less than twenty years old, and practically useful chatbots are less than five years old. The emphasis is therefore on learning how to teach yourself to learn to use useful computer based tools.

These notes are written as a series of `qmd` files. This makes it a Quarto book. Quarto can be seen as a successor to RStudio and `Rmd` files. But rather than focusing on Quarto or `qmd` files per se, what I hope people will take away is that plain text files are enduring, human readable, and easily shareable. Computers can help with all the complicated syntax, and that is where **markdown** variants are useful. `qmd` files are one such variant, and Quarto is the tool that takes our plain text and turns it into something pretty. First, we wrote in $_{\text{notepad}}$, then we used RStudio, and now we are using Quarto (which you can use in RStudio, but which we will use from inside VSCode). What will you do when Quarto is passé? Or a collaborator wants to use a different markdown variant? You need to be able to understand how to change your syntax, download and install the new tool. That meta-knowledge should be your real goal in this course.

The subsequent material is generally ordered in the way I intend to teach it. These notes are not self-contained, but are intended to be reference material. If I mention a tool in class this is where you can find the name and link if you did not catch it. I may also demonstrate tool use in these pages and there will often be code that you can cut and paste to try things out. Think of this book more as a "student's guide" than as a textbook.

Chapter 1

Introduction

This is a book being created for PSYCH390 at the the University of Waterloo.

It is very much a work in progress. You can help it get better by providing feedback. Ideally you will do this by raising an issue on the github repo. Even better is if you would fix the book yourself and make a pull request.

Chapter 2

How can a computer help us in our research?

> Class Question
>
> What are steps (or stages) involved in psychological research?

2.1 Research Project Support

Knowledge and Literature Management.

Telling the Computer What to Do

Managing Data.

Analyzing and Visualizing Data.

Sharing Your Findings.

Sharing Your Data

2.2 Open Research Practices

> Class Question
>
> - What is "open research"?
> - Should research be open?

10 strategies for open science

Class Question

What are the **Fair** principles for research?

- Fair principles
- Fair guiding principles

Read and discuss: A toolkit for transparency ….

Chapter 3

IDEs

- Who are you writing code for? Human or Machine?
- What is an IDE? What makes for a good IDE?

3.1 Required for this Course: VS Code

VSCode

Opensource alternative

3.2 For the Hardcore

Emacs. VIM

Chapter 4

Using VSCode

https://www.youtube.com/watch?v=B-s71n0dHUk

- Basics video
- Using VSCode with R
- Using VSCode with Python

Chapter 5

Jupyter Notebooks

An alternative environment for analyzing and visualizing data.

https://www.youtube.com/watch?v=suAkMeWJ1yE

Chapter 6

Terminals

Currently these words are used as synonyms: terminal, shell, command line; which is not exactly true, but is close enough.

https://vimeo.com/453837142

- Power Shell.

- OSX Terminal

6.0.1 Commands to try (many have options)

- `ls`
- `ls -alt`
- `cd`
- `pwd`
- `mkdir`
- `rm` **DANGER**
- `mv`
- `cp`
- `cat`
- `less`
- `find` **very powerful**
- `du`
- `df`
- `touch`
- `ln`
- `chmod`
- `chown`

6.1 Additional Introductory Material

1. The command line
2. A Short Series of Terminal Lessons from the UbuntuWiki
3. Some Scripting Basics
4. Another Scripting Introduction

6.2 Getting work done: scripting

Want to convert and compress a large directory of videos (as I did for the pandemic version of a course)?

```
for i in *.MP4;
do name=`echo "$i" | cut -d'.' -f1`;
echo "$name";
ffmpeg -i "$i" -c:v libx264 -b:v 1.5M -c:a aac -b:a 128k "${name}S.mp4";
done
```

6.3 Talking to other computers in the terminal

6.3.1 SSH

Testing and Learning for Linux SSH

Chapter 7

Version Control

7.1 Git

7.2 An old overview of git and Github

https://vimeo.com/456349738

7.2.1 Getting Git

It depends on what system you have. If you are using wsl then you are using linux. Additional installation instructions.

7.2.2 Configuring Git

Do this in your terminal

```
git config --global user.name "John Doe"
```

```
git config --global user.email johndoe@example.com
```

7.3 Other version control system

1. Mercurial
2. Darcs
3. CVS
4. Subversion
5. Pijul

7.4 Git is not Github

And github is not the only way to share code and data. 1. OSF.io 2. Sourceforge 3. Bitbucket 4. Gitlab The university provides you with a gitlab account: https://git.uwaterloo.ca 5. Codeberg

7.5 Using git with github

You will need an ssh key.

The offical instructions

7.5.1 Creating an ssh key with your terminal

Detailed instructions on ssh-keygen

A slower walk through with some github specifics

7.6 Understanding Git and the Workflow in Pictures

Git can be confusing.

Clone or fork?.

A long video where I try to talk and draw through the concepts. https://vimeo.com/456349595

7.7 Principal Terms Terms for Git

- **Fork:** A copy of one github repository to another **github** repository.
- **Clone:** A copy of one **git** repository to another **git** repository.
- **Remote:** This is a repository that you are following. You will typically *pull* from these, but your *push* permissions may be limited depending on the distinctions between forks and clones, and whether you own the remote or someone else does. You can have more than one remote.
- **Pull:** the transfer of information and changes **from** one repository incorporated into another. This is how you get the new information from a remote transported to a local repository that you control.
- **Push:** this is the transfer of information **to** a repository you control (or have permissions to push to) from another repository that you control. This is often from your local laptop version to the hosted repository (your fork) on github.
- **Pull Request:** When you have information or changes that you think would be helpful to a remote you do **not** have push permissions for then you can request that the owner of that repository pull in your changes.

This is a formal process called a pull request. It is primarily a github concept and not a git concept.

- **Branch:** within a repository the development of the code may be proceeding in a few different directions at the same time. The principal production branch is conventionally called **master**. And the principal repository that is the main, shared one is called **origin**. We will not be working with branches in our course, but those terms do show up in commands.

Chapter 8

Lab Notebooks

I will not actually be spending much time on this, but we could. It is a neglected practice in psychology to keep a lab notebook that is updated. Here are some options if you are interested in exploring this topic.

- Jupyter notebooks in Psychology
- 10 rules for lab notebooks (electronic)
- How to keep a lab notebook
- Pick an electronic lab notebook
- Lab notebook 2023 guide

Chapter 9

Writing Scientific Documents

Writing scientific documents is more than writing papers. It can also include posters, websites, books, and all of these may require images, graphs, references, formuals, and even code. We will take a look here at some of the tools that can help with this, and later we will look in more detail at some of the options.

9.1 IDEs and Writing up your Research

Class Question

What does IDE stand for and what are examples of IDEs?

9.1.1 VSCode

There are many powerful IDEs available. It will be useful for you to try more than one. However, at this moment in time the clear winner of the IDE popularity contest is Microsoft's Visual Studio Code.

Since I want you to learn to use your computer as a tool, and to continue to be able to do so when tooling changes, you need to be able to:

1. Locate code/programs on the internet
2. Download and install that program to your computer
3. Locate the help to begin to use the tool without formal instruction.

In-class Exercise

Locate, download, install, and verify you can start VS Code.

9.1.2 Some helpful links to get you started

- Basics Video
- Using VSCode with Python
- Using VSCode with R
- Text Editor Page from Quarto Site
  - What is "pip"? And why will you need to know?

Additional material about VS Code and its use as an IDE will be found in this section quit ### Overleaf

VS Code is far from your only option. For technical writing and collaboration many scientists rely on Overleaf.

Nature will [let you submit your manuscript](https://www.nature.com/srep/author-instructions/submission-guidelines) from an Overleaf template.

You always have choices. You can even [write LaTeX with Quarto](https://github.com/James-Yu/LaTeX-Workshop?tab=readme-ov-file) (see also).

Classroom Discussion

What is single source publishing? How does this idea impact your preference for tools like Overleaf and Quarto?

9.1.3 RStudio

RStudio is a popular IDE for writing R code. R is a programming language particularly useful for statistical analyses. While initially RStudio was only for the R language the number of programming languages it currently supports has grown. It still is not as widely useful as VS Code, and many of RStudio's features are designed for the interactive evaluation of data, and so it has less of the cutting edge features that software developers expect in their code tools. Whether you want to use RStudio or not will depend in part on your planned activity.

9.2 Scientific Publishing Tools

To understand the plethora of contemporary tooling available to facilitate writing you should know generally what is meant by a mark-up language. This will help you evaluate whether it makes sense for you to switch from a word processiing program, like Word for Windows, to a more general tool like VS Code or emacs.

Some of the available options you should be familar with are

- LaTeX
  - LaTeX This was the standard for several decades, and used more by mathematicians than others, but today it seems limited in that it produces mostly pdfs, and many people also want their files to export to html (webpages)
- A guide to LaTex for Word Users (pdf)
- The descendants of markdown
- Quarto/Qmd
  - Quarto (we will skip Rmd and since we are using its successor qmd. Both are related to the people who wrote RStudio,`knitr`, and are now known as Posit).
- Org Mode
  - Org Mode works particularly well with Emacs and allows you to embed code from multiple languages.

9.3 Things You Can Do With Quarto

Here are some of the helper links for things we can do with Quarto:

- journal formats try the Elsevier format for this course. A lot of psych journals are Elsevier owned.
- presentation
- Talking to databases.
- Make a website/blog for your work or lab
- And more …

9.3.1 Making a Website with Quarto and Github Pages

As science progressively moves to online communication there is a need to have a place to show case your work and share ideas. One way to do this is with a personal website structured around your scientific interests and activities. While there are many methods, tools, and hosts for this I will focus on sharing how you can use the Quarto publishing tool.

9.3.1.1 Hosting

To have a website viewable by the world it must be hosted on a server that is viewable by the world. For example, it cannot be behind an institutional firewall (as are most of the computers at the University of Waterloo). And if you host on a University or Company server you have the potential of being limited in what you can share, how often you can update, or you may be required to adhere to certain styles and formats mandated by the institution's brand. One way to avoid this is to find an external hosting site. A modest fee would allow you to have a virtual server in the cloud, but the example we will use here to get you started is a free service available to those with Github accounts: Github Pages.

9.3.1.1.1 Testing Github Pages

Assuming you have an account you should be able to create a minimal webpage in a few steps.

1. Go to personal dashboard and create a new repo as instructed on the Github Pages website. The repo must be named .github.io.
2. Use terminal to clone this repo to your local computer.
3. cd into the correct directory using your terminal (inside VS Code is fine). And using the *terminal* do 'echo "hello world" > index.html.
4. Do the usual add, commit, and push and verify your web site works at <username.github.io>.

You now have a website!

9.3.1.2 Blog Project in Quarto

The advantage of managing your blog (or website) in Quarto is that you get to take advantage of all the markdown language features. This is much more convenient than writing in html directly. You can also embed code and output just as you have been doing when you export your R code to html.

There are several ways that the Quarto people recommend getting the material to and from github. The one that I found most convenient (that is the one that I got working with the least amount of swearing) was to configure a project that exports to a `docs` directory.

Creating the blog project in Quarto.

1. In the `terminal` move to the directory with your repo.
2. Create a branch called `gh-pages`.
    1. You can find the steps for creating this branch at blog, but for the later steps he uses the publish option where we will output to `docs`.
    2. Make sure everything is committed or stashed before you begin then:

```
git checkout --orphan gh-pages
git reset --hard
git commit --allow-empty -m "Initializing gh-pages branch"
```

> Caution
>
> If you are going to try the "publish" option you will not want to edit in the `gh-pages` branch. That branch will just be used for publishing your content. However, when using the `docs` approach, as I am illustrating, it is fine to just work on the `gh-pages` branch directly. For the `docs` method to work properly you must go into your settings for this repository on Github and find the Pages area. Then make you set the branch correctly and point it to the `docs/` directory.

3. Add the following to your .gitignore:

   ```
   /.quarto/
   /_site/
   ```

   If you don't have such a file, create it. If you don't know what a .gitignore file is ask!

4. Now we use the quarto command in the terminal to initiate our website as a project. This creates the basic structure we need. We can later change or adapt it.

```
quarto create project blog .
```

When I did this it asked me for a title and then it asked to open everything in vs code. Then you will want to edit the `_quarto.yml` file to direct the output to the `docs/` directory. The top of you file should look like

```
project:
  output-dir: docs/
  type: website

website:
  title: "brittAnderson.github.io"
...
```

5. Try `quarto preview` to see the current skeleton.
6. Explore the files to see what you can change. Try the about page for a starter. Can you add a picture of yourself, and make it say your name?
7. Try adding a post. First study the structure of the existing posts. You will need to emulate this in order for things to track correctly.
8. Then publish `quarto render` This command will compile the html versions of your qmd code and direct them to the correct places. Then if you have set up thing correctly you can use the git features within VS Code to commit and push your changes. After a few minutes they should be available via your github link.
9. My files are at Britt's Github Example and the demo is at brittanderson.github.io.

Note added Sept 2025. A good article on academic websites is to be found here: https://doi.org/10.1038/s41562-025-02310-6

9.3.2 Journal Article Authoring

Classroom Exercise and HW

Write an article with quarto/qmd and using any of the linked article formats. Each has instructions on their use. Your article should look similar to your overleaf submission. All the components of an article, at least one figure, and at least one reference. Instead of using an image from the web try to include either your R or Python code for the figure you liked.

I find this easier to do from a terminal.

```
#| eval: false

quarto use template quarto-journals/elsevier
```

This will result in several questions for you to answer. When you are done you have starter versions of all the pieces you will need to get going.

9.3.3 Presentations

Presentations are a standard component of professional life. Despite that importance the materials are often developed in a casual way that does not give the best impact to the material. There are many more versatile tools than contemporary Power Point. Though power point can make an effective presentation its default use does not typically achieve that goal. Quarto provides easy access to two other presentation tools that will expand one's recognition of what an effective presentation tool can do.

9.3.3.1 Reveal.JS

Reveal.js is a javascript library for the creating presentations that work as well on the web as they do in person. Quarto also supports Reveal.js.

9.3.3.2 Beamer

Beamer is a package for LaTeX that allows professional quality typesetting. It is particularly relevant for presentations that have a lot of mathematical material, though it is quite capable of supporting any general presentation as well. It is less flexible for the web since it produces PDFs. These can be shared on the web, but they require the intermediate step of downloading them rather than just playing them. Beamer also has a "Poster" variant that can be used to make high quality academic posters.

Overleaf has a beginners tutorial. Quarto also supports Beamer.

LaTeX and Knitr in Overleaf][knitr]] with overleaf. A scientific poster with Overleaf?

9.3.3.3 Academic Posters

Typest

[Scientific Posters with Quarto](https://github.com/quarto-ext/typst-templates/tree/main/poster)

Chapter 10

Coding

For a lot of the above we need to know how to code. In order to write code we need minimally to answer two questions. What language? What tooling?

10.0.1 Languages

What should I consider when selecting a programming language? Will it do what I need it to do now and tomorrow.

- Is SPSS a good language for statistics?
- Is R a good language for statistics?
- Is Python a good language for statistics?
- Is R a good language for coding a web app?
- Is R a good language for coding an in-lab visual experiment?
- Should you use Julia? Common Lisp? Haskell? Lean? OCaml? Rust? Go?

How do you *future proof*? - If languages go in and out of fashion what is it you should really be learning about programming? What are good coding practices?

10.0.2 Programming Languages

10.0.2.1 Basic Constructs

- Variables Variables are things (lists, values, names, and such) that you want to give a name to. You might do this because you think you are going to want to do something with them later and it might be more convenient to have a short, memorable name for referring to them.
- Functions These have inputs and output. The inputs may be called parameters. Sometimes the outputs are called return values. What the function does is describe the processing of the inputs to the outputs. Think

  of a factory. It takes in raw materials (inputs) and produces an output. You are describing a factory when you write a function.
- Interpretation and compilation.

10.0.2.2 R

10.0.2.2.1 Installation

- OSX
- Windows
- Linux

Download the CORRECT R

Follow the appropriate instructions for your operating system.

If that went well, you need to make sure you restart VS Code and then click on the little set of squares at the left to install the R extension for VS Code. If you are using VS Code for your IDE as recommended.

If that goes well check out the book on using R that is itself written in Quarto, by the programmer who authored the tidyverse (see below). And make sure you can create a file in VSCode as a quarto document, cut and paste in some R code, and see the whole thing compile to a web page.

10.0.2.2.2 Working Methods - Generally

Have your code open in a window and a terminal open running the R interpreter for testing and checking.

Test your system by, 1. `cd` to the directory where you want to work. 2. open a .R file in your editor. 3. open a terminal that points to the same directory you can move with `cd` and check your location with `pwd`. 4. Verify that you can open R there by typing `r` and then hit enter. If you need to quit you can type `quit()`.

10.0.2.2.3 Packages/Libraries

Many programming languages are very basic in what they offer. In order to do the cool stuff you will need additional collections of code, some of it may even be written in a different programming language, often called packages or libraries that you import, source, or load. `ggplot2` was an example of that for R and `matplotlib` for python.

```r
install.packages("tidyverse",repos='https://mirror.csclub.uwaterloo.ca/CRAN/')
library{tidyverse}
```

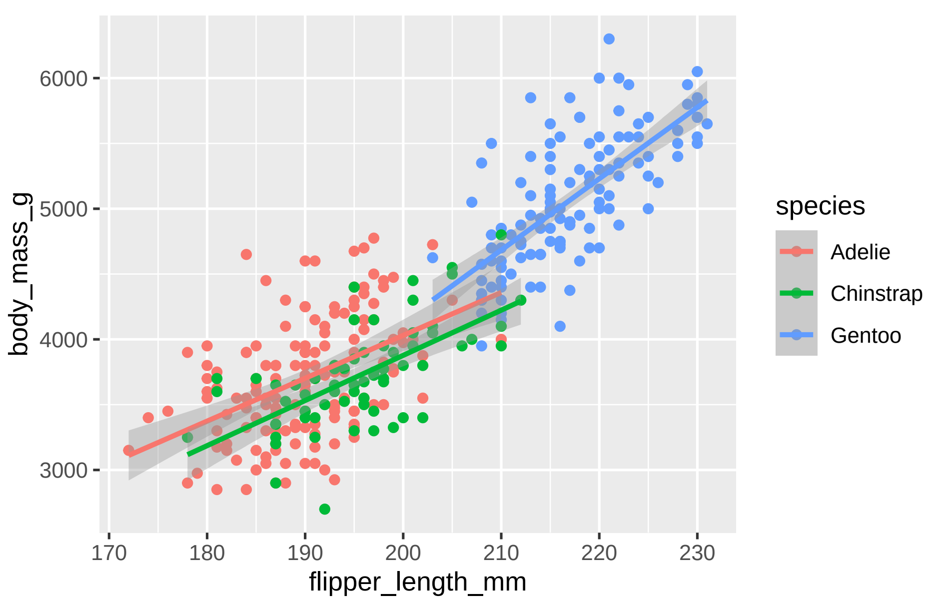
Figure 10.1: This is an example of installing a package in R and then using it. Note the inconsistent use of quotation marks.

The tidyverse is a very popular collection of R code that itself will depend on many additional pieces of R code and other packages, some R and some not. I personally often try to just use “base plot”, but most people seem to prefer the “tidyverse”.

Here is an example of embedding R code in a qmd document. This is very useful when you get to the stage of data analysis or writing your research report. But when you are first exploring the data it is often better to work directly in an R environment and record your code in an .R file to make it easy to replay and edit your work as you go.

```
if(!require(tidyverse)){
    install.packages("tidyverse",repos='https://mirror.csclub.uwaterloo.ca/CRAN/')
}
if(!require(palmerpenguins)){
    install.packages("palmerpenguins",repos='https://mirror.csclub.uwaterloo.ca/CRAN/')
}
library(tidyverse)
library(palmerpenguins)

ggplot(
  data = penguins,
  mapping = aes(x = flipper_length_mm, y = body_mass_g, color = species)
) +
  geom_point() +
  geom_smooth(method = "lm")
```



10.0.2.2.4 Types

Many programming languages have a notion of a datatype. `1` can be a number, but "1" can also be a character that you are typing. It doesn't make sense to add one, the number, to one, the character, but it might make sense to `1 + 1` for either. You would expect 2 for numbers and maybe "11" for characters. R in particular has many complicated data types that represent the output of functions and statistical analyses. If things are getting confusing you might want to check on the type of your data.

```
a = 1
typeof(a)
[1] "double"
```

Figure 10.2: Checking the type of a number in R.

Other data structures that you might run across are: - Lists, - Tuples, - Data.Frames

```
tpl = c(1,2)
lst = list("firstName" = 'Britt', "lastName" = 'Anderson')
df = data.frame('fn' = c("bob","jane","griffin"),"gndr" = c('m','f','o'))
df
       fn gndr
1     bob    m
2    jane    f
3 griffin    o
```

Figure 10.3: Each of three datatypes in R are demonstrated. Think of data.frames as a supercharged spreadsheet with columnn names.

10.0.2.2.5 Data.Frames

`Data.Frames` are particularly central to the R analysis model and are a killer feature. They are also moving into the python analysis space. Let's look at the cars data frame that comes as part of the base R installation. You can verify it is installed like this.

```
is.data.frame(cars)
```

```
[1] TRUE
```

```
head(cars)
```

```
  speed dist
1     4    2
2     4   10
3     7    4
4     7   22
```

```
5       8    16
6       9    10
```

10.0.2.2.6 Selecting Data

A bit of code that tests for whether something is true or false can be called a *predicates*. These are important when having your code do something conditionally. If this is true then do this otherwise do that. This common construct of "if-then-else" exists in most programming languages.

```
length(cars$dist[cars$speed > 10 & cars$speed < 20])
[1] 29
```

Figure 10.4: How many cars are there that can go faster than 10, but not more than 20?

How would you do that in Excel?

What is going on here? 1. `cars` is the name of the data frame. 2. We access a *column* of the data frame with the dollar sign notation `cars$dist`. You can see the names of the columns in a data frame with `names(cars)`. 3. We use the square brackets to *index* the column of data we are interested in. Here we do not use specific numbers, but rather we use a *boolean* to compare the values in a column and we only include the *TRUE* values. Here we ask for all the indices where the speed is greater than 10, but less than 20, and we use those indices to get the values for the `dist` column.

Practice

1. Sort (or $_{\text{order}}$) cars by the $_{\text{dist}}$ variable.
2. Find the mean and standard deviation of the speed of the cars.
3. Are there other datasets?

   ````
   ```{r}
   library(help="datasets")

   ```
   ````

4. Open any of the datasets that catches your eye.
5. What are the column names?
6. How many rows?
7. What is the *comment* designator for R?
8. What is the ending extension of an R script?

10.0.2.2.7 Loops

Tip

Loops are very different between python and R. You use them a lot more in python than R. R often allows you to "map" over data.

`For` loops are the most common looping construct.

```
ml <- seq(1:10)
for  (m in ml) {
  print(m)
}
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
[1] 6
[1] 7
[1] 8
[1] 9
[1] 10
```

Another kind of loop is the `while` loop.

Test your understanding with more practice.

1. Edit the above so that it prints the individual number each time it goes through the loop, and not the whole list.

Another project 1. Create a list of at least 8 individual characters. 2. Make sure they are **not** in alphabetical order 3. Print the letters one at a time. 4. Print the letters sorted alphabetically one at a time, but *do not* overwrite your original list. 5. Print the letters from both lists with a `format` command that says which position the letter is in.

```
myName = "brittAnderson"
myList = unlist(strsplit(myName,""))

for (l in myList){
  print(l)
  }

for (l in myList[order(myList)]){
  print(l)
  }

i = 1
for (n in order(myList)){
```

```
  t  <- sprintf("The %.0fth letter of myList is: %s, but is %s in the sorted list.",i,myList[i],m
  print(t)
  i = i+1
  }
```

10.0.2.2.8 Examples of Functions

```
myadd  <- function(x,y) {    ①
  return(x+y)                ②
  }
```

① The arrow is used to assign something to a name. Other languages usually use the equals sign. The keyword `function` tells R you are creating a function, and the parentheses following establishes the number of inputs and the nicknames you will use in your function.
② `return` specifies what we want to get back from the function.

Now we can use the function we defined with particular inputs.

```
myadd(2,3)
```

```
[1] 5
```

A few years ago I made a video talking about some of these features, but I used a different IDE from VS Code.

Hc : Walking Through the R Topic (i2c4p) from Britt Anderson on Vimeo.

10.0.2.3 Python

> **Note**
>
> If the following doesn't go smoothly for you check out the additional suggestions in the python-venv appendix.

10.0.2.3.1 Installation

Installation Instructions

Install the `mathplotlib` and `numpy` packages.

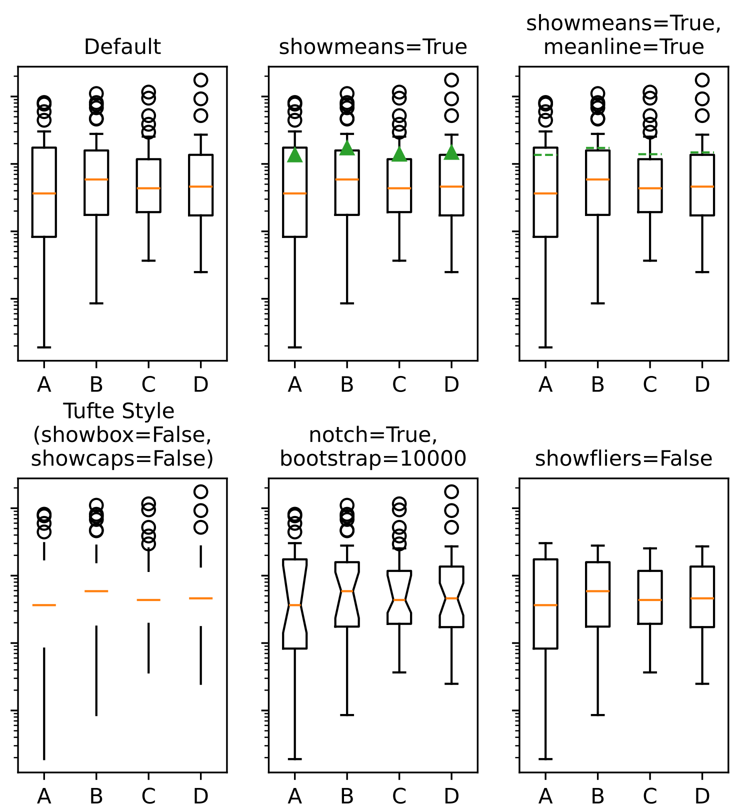
> **Class Discussion**
>
> What is a "package" when talking about programming languages? What is a "library"? What is an "executable"?

> ⚠ Warning
>
> Ask me about virtual environments!

setting up a venv in vscode and python

10.0.2.3.2 Testing

> ⚠ Warning
>
> You may need to install the "reticulate" package for R if you want to run both python and R code in the same document as I am trying to do here.



> ⚠ Warning
>
> Python cares about whitespace. It is strongly recommended that you only use spaces and not tabs when writing python code.

```
my_name = "britt"                          (1)
my_name_as_list = list(my_name)            (2)
for i, j in enumerate(my_name_as_list):    (3)
  print((i, j))
```

(1) Assignment in python does not use the arrow like R does.
(2) Strings and lists are different *types* of data. I have to coerce my string to a list.
(3) Python has conventions to make it easy to define multiple looping variables at once.

```
(0, 'b')
(1, 'r')
(2, 'i')
(3, 't')
(4, 't')
```

Figure 10.5: What a for loop might look like in python.

10.0.2.3.3 Some programming vocabulary

If you want to watch a video that reviews some of the programming vocabular and examples typed out (using Emacs and not VS Code) the following video may be of use.

Gd : Common programming terms (i2c4p) from Britt Anderson on Vimeo.

An overview of a few basic and fundamental programming terms.

> ❗ Important
>
> Python is a dynamically typed language. Python will try to guess the type of your data or change it based on context.

10.0.2.3.4 Assignment and Equality Are Different in Programming

Assignment means you are assigning a name to something. Equality is that you are testing if they are the same. One equals sign does not equal two equals sign. That is `= != ==`. Exclamation points are often used to *negate* something (turn true to false and vice versa).

```
a = 2
print(a == 3)
```

```
False
```

For loop details in python

Python refers to things called "iterables." To iterate is another way of saying something you can keep doing the same thing over and over to. Imagine a bowl of ice cream. It is "eatable". You take one spoon, and keep taking spoonfuls until the bowl is empty.

Indexing is how you get the location of an element in a list. Think of the row or column number in a spread sheet program, but indexes can be more powerful, and can be nested easily. In many programming languages, but sadly not all, **indexes start at 0**. Different programming languages will have slightly different syntax.

```python
name_dict = {'firstName' : 'Britt', 'lastName' : 'Anderson'}
my_list = list(range(1, 10))                                          ①
print(name_dict['firstName'])
print(my_list)
print(my_list[0])
print(my_list[-1])
print(my_list[0:4])
```

① Notice that here we use `range` where we used `seq` in R. There is usually a way to do something similar in both languages, but they may not have the same name.

```
Britt
[1, 2, 3, 4, 5, 6, 7, 8, 9]
1
9
[1, 2, 3, 4]
```

Figure 10.6: The use of the `-1`as an index is a python trick for getting the last element in a list. Think of the list as a circle. If you count backwards from a list you will get to the beginning eventually (index 0) if you went back one more step (-1) you would circle back to the end of the list. Test what happens when you try -2. Does it keep going? A lot of learning how to program is just doing stuff to see what happens.

A `for` loop in python

```python
for ml in my_list:
  print(ml)
```

```
1
2
3
4
5
```

```
6
7
8
9
```

```
for i, ml in enumerate(my_list):
  print("The {0}th element was {1}".format(i, ml))
```

```
The 0th element was 1
The 1th element was 2
The 2th element was 3
The 3th element was 4
The 4th element was 5
The 5th element was 6
The 6th element was 7
The 7th element was 8
The 8th element was 9
```

Conditionals in python

```
if 2 == 3:
  print("Wha.....?\n\n")
elif 3 == 2:
  print("Now that is odd")
else:
  print("2 does not equal 3.")
```

```
2 does not equal 3.
```

Note that we use colons and white space to organize code in python, not the {} of R.

> **!** Important
>
> Python uses a different syntax to define functions from R.

```
def my_add(x, y):
  return(x + y)
```

10.0.2.3.5 Popular and Useful Python Libraries

1. Numpy
2. Scipy
3. Matplotlib
4. Pillow
5. Sympy

10.0.2.4 Javascript

10.0.2.4.1 Installation

Since javascript is already included in most browsers you likely have a basic version of javascript installed, and you can get started with almost no additional tools. As a quick illustration open up your favorite browser and look in the settings or options for something like "more Tools". Somewhere in that menu should be a web tools or web developer tools entry. Click it, and you should see something that looks like your VS Code screen with terminal like objects at the bottom. One of those is the *console*. Select it. Then you can create a function and use it with something like:

```
function add(a,b=10){return a + b}
```

A simple javascript function.

10.0.2.4.2 A Better Way to Go

Even though there is a convenience factor in using your browser's console it is a very limited way to code. It is fine for testing out a line or two of javascript, but to actually code something you will want an actual development environment, and we will use VS Code.

To code javascript outside the browser you need a proper environment such as Node.js. There are many ways to install this, but if you are new to javascript using the official installer is probably best. To check if the installation went well you can run `node -v` inside a terminal. You should see a version number. If you are sure you downloaded things correctly you may have to set your path so your terminal can find the installation.

To test your installation you can create a test folder, e.g. call it "hello" and open that folder in VS Code. Then create a new file "app.js". VS Code should give you a lot of help out of the box as you enter:

```
var msg = 'Hello World';
console.log(msg);
```

After saving you should be able to move down into your terminal and run `node app.js` to then see "Hello World" printed.[1]

[1] This example comes from this node tutorial.

Chapter 11

Programming Experiments

One of the most common uses of computers in psychology is for the programming of experiments. Classic experiments often involve a combination of visual stimuli that the participant has to respond to in someway. Typical responses are often button presses or mouse clicks, though many other varieties of stimuli and reports are used. While many programming languages can be used for this task we will begin with one of the most popular, Python. And if time permits we can explore some examples with Javascript as the world of experimental psychology is increasingly complementing the usual lab based testing paradigm with online studies.

11.1 All-in-one options

Excellent tools exist that provide turn-key solutions. A very popular one is Psychopy. This package is available for the popular operating systems and gives a gui like builder that allows you to program visually. It offers the opportunity to output your experiment into a web friendly format that can integrate with an online hosting service for performing experiments. However, my experience is that while such tools make it easy to get started they don't help people develop the general skills that will transfer into programming that distinctive experiment that no one else has done. And the knowledge you gain is often interface specific. It is not general, and so your learning cannot transfer into writing R code or porting your experiment to MATLAB if that becomes the tool a collaborator wants to use. Even if this is the sort of tool you eventually settle on for the very reasonable reasons that your experimental needs are not complex and the quality of the implementation allows you to get work done more easily and on board new investigators more quickly, it will still make you better at planning and handling problems if you have a knowledge of what is going on under the hood.

11.2 Python

The following examples focus on the sort of experiment where you want something to appear on the screen, and you want feedback from the participant. Current computer graphics don't work like they did twenty years ago when we all had heavy CRTs on our desks, but one can still deploy some of the intuitions of that era.

The monitor is a screen and it is displaying a picture. In the background what is to be shown next is in preparation and at some point what is on the display now will be replaced by the next collection in the queue. Even though we no longer literally raster the display from top to bottom and side to side you can still usefully think in terms of *frames*. Thus, almost all your experiments will need the mechanics of a window the participant views and in which you can write things you want them to see.

11.2.1 Pygame

For practice I will use the pygame python library. There are a large number of help and tutorials for this well established library that has been in use for decades. It is well vetted and documented, and relies on a simple direct library for video manipulations.

11.2.1.1 Some Links for Getting Started with Pygame

1. The pygame getting started wiki. Most of my examples will draw from this site.
2. Real python pygame tutorial
3. Game Development Academy

11.2.1.2 Installation

A virtual environment is recommended, but not necessary. If you are using a venv activate it first.

Then

```
pip install pygame                                    ①

python -m pygame.examples.aliens                      ②
```

① This line installs the library.
② This launches a test game. If you see the game, you have installed pygame. I find this game plays a little fast on modern hardware.

One of the better ways to get started is to gradually hand in functionality one step at a time until you have the basics of what you need on a trial and then to refine that to loop through the multiple trials you will probably need.

```
import sys,pygame
pygame.init()

mywindow = pygame.display.set_mode([800,800])
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
    mywindow.fill((128,15,15))
    pygame.display.flip()

pygame.quit()
sys.exit()
```

Figure 11.1: This python program will, assuming you have installed pygame, launch a window and close it when you click the "x". Can you change the window size and background color?

Here is an example of a bit of code that shows some elementary use of the mouse, response time, shape and text functionality.

```
from itertools import compress
import sys,pygame
pygame.init()


my_font = pygame.font.Font(None,72)
test_text = my_font.render("Hello P390", 1, (200,10,200))
test_text_pos = test_text.get_rect(centerx = 400)

#Helper functions
def which_buttons (bs):
    return(list(compress(range(len(bs)),bs)))

#button colors
clrs = [(0,0,255),(0,225,0),(225,0,0)]
actv_clr = 0
rslts = {'trial_num': -1,'color' : None ,'time' : None}

mywindow = pygame.display.set_mode([800,800])
running = True
myrt_clock = pygame.time.Clock()
t1 = myrt_clock.tick(60)
anew = pygame.mouse.get_pressed()
trl = -1,
```

```
clr = clrs[0]
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
    mywindow.fill((128,15,15))
    pygame.draw.circle(mywindow, clrs[actv_clr], (400,400), 75)
    mywindow.blit(test_text, test_text_pos)
    pygame.display.flip()
    aold = pygame.mouse.get_pressed()
    if(any(aold) and (aold != anew)):
        t2 = myrt_clock.tick(60)
        rslts['time'] = t2 - t1
        rslts['trial_num'] = rslts['trial_num'] + 1
        actv_clr = which_buttons(aold)[0]
        rslts['color'] = actv_clr
        anew = aold
        print(rslts)

pygame.quit()
sys.exit()
```

11.2.1.3 Next Steps

You will want to save your data. As a first step see if you can open a file, and the if you can use the dictionary created to add lines to a csv file.

11.3 Javascript

WIP

Chapter 12

In Lab Experiments

- Making stuff appear on monitors.
- What is OpenGL?
- Use pygame to make some simple visual experiments.
- Have a beauty contest the following week to see what people have been able to make?
- Make sure that people are using venv

12.0.1 Online Experiments

Running a server for testing and more? XAMPP Running a lab now seems to require some familiarity with servers. And people who want to write their experiments in javascript often want to try things out first so it seems something like XAMPP might be a good resource. *Exercise* to download and get the XAMPP server running? Need to expand this. An exercise with JavaScript? This site has a simple bit of code for throwing an image on the screen. Then use the XAMPP server to test it? Require changing the image? Animate a button to toggle or get a random image?

Chapter 13

Handling Data

Before we can analyze data, we have to have some data. Broadly speaking our data may come from experiments we run ourselves, or it may come from data that is shared with us. In the first case we have some freedom to choose the format. In the later, we simply have to take what we are given. Therefore, we need to consider both how to save and convert data that we collect from an experiment. And we also need to be broadly familiar with common formats of data storage that others may use.

13.0.0.1 Data Formats

- Plain Text
- CSV
- TSV
- JSON
- Pickle

- Dat (R)

Some advantages and disadvantages of each.

Plain text is easy to do and human readable. It usually will require a unique parser, and forces all items to be text. Thus, things like numbers lose their specificity, and things like lists can become painful to retrieve.

CSV and TSV view all data as rows and a unique character (commas or tabs) separates the cells of the rows. Another character, typically a new line (often seen written as `\n`) is what distinguishes one row from the next. These file types are, largely, human readable if the number of columns and rows are not too large. They are not as space efficient as other formats, but that may be a reasonable tradeoff when the size of the data files is small.

JSON stands for Java Script Object Notation. Although, as the name implies, it had its origin in javascript it is largely language agnostic and is a common format for the storage of data of all types. Its being used as a standard means it has wide support from many programming languages and that means there are good tools for exporting and storing for most programming languages. It can be quite flexible in the type of data stored. It is not friendly for writing by hand, but the use of tools makes it fairly easy to take experimentally collected data created in, say, a python dictionary and convert it to a json format.

```
import random as r
import json as j
a = {'name': 'John Doe', 'age': 24, 'rt': r.random()}
js = j.dumps(a)
js
'{"name": "John Doe", "age": 24, "rt": 0.43681265179099293}'
```

Figure 13.1: The `dumps` command will output the json version as a string.

Note there is a very similar python function without the terminal *s*, `json.dump`, that will save the json data to a file. Some other code is needed to create the file pointer. But once that is done, and the json data saved, you can read it back with `json.load()`

```
import json
a = {'name': 'John Doe', 'age': 24}
js = json.dumps(a)

# Open new json file if not exist it will create
fp = open('test.json', 'a')

# write to json file
fp.write(js)

# close the connection
fp.close()
```

Figure 13.2: A simple example taken from [StackOverflow] shows a working example.

A JSON tutorial can be found here.

Pickling is a data serialization library specific to the python ecosystem. That makes it useful if you are coding your experiment in python, but not in any other language. However, it can be very efficient on space, and there is good support for it in the python language so many experimental coding libraries built around python use pickle for data storage.

This code from a GeeksforGeeks article provides a nice minimal working example.

```python
import pickle

def storeData():
    # initializing data to be stored in db
    Omkar = {'key' : 'Omkar', 'name' : 'Omkar Pathak',
    'age' : 21, 'pay' : 40000}
    Jagdish = {'key' : 'Jagdish', 'name' : 'Jagdish Pathak',
    'age' : 50, 'pay' : 50000}

    # database
    db = {}
    db['Omkar'] = Omkar
    db['Jagdish'] = Jagdish

    # Its important to use binary mode
    dbfile = open('examplePickle', 'ab')

    # source, destination
    pickle.dump(db, dbfile)
    dbfile.close()

def loadData():
    # for reading also binary mode is important
    dbfile = open('examplePickle', 'rb')
    db = pickle.load(dbfile)
    for keys in db:
        print(keys, '=>', db[keys])
    dbfile.close()

if __name__ == '__main__':
    storeData()
    loadData()
```

When working in the R environment the `save` and `load` function provide convenient ways to save your data and variables in their current state so that you can make available in your next interactive session what was available in your current one. However, this may not be the most interoperable way to work with your data if you need to share with collaborators who may not use R or if you yourself are open to the possibility of changing your analysis software. R offers two main functions for reading and writing data called `read` and `write`. Each of these has several specialization that target particular file types (e.g. `read.csv`) and also allow a number of customizations (e.g. whether to ignore a number of lines at the top of a file if there is a plain text header). Usually one of these variants will allow you to read your data into R or write data to a file that can be read by others.

13.0.0.2 Considering Pandas and Interoperability with R

Pandas is a major python library and effort to provide cutting edge tools for statistics and analysis to match what is on offer elsewhere. In my experience they are not the first place that statisticians develop their cutting edge tools, R remains most popular, but for all else the choice between the two, R and Python, is really a matter of preference and comfort. Pandas is a very powerful tool and if you are more likely to use python in the future you will want to explore Pandas fully.

Pandas can usually be installed with `pip install pandas`. Because of the size and complexity of python installations a virtual environment is recommended.

Even if you are not thinking of doing your data analysis in python if you are writing your experiments in python you might want to take advantage of some of the pandas tooling to make saving your data convenient. But before we show a pandas method let's see a lower tech, more old school method.

It is possible to manually write a csv file in python using a print command where you explicitly coerce everything to a string and intercalate commas along the way. For example,

```python
a = 1
b = "britt"
c = 2.3
mycolumnheadings = "a," + "b," + "c" + "\n"
mystring = str(a) + "," + b + "," + str(c) + "\n"
print(mycolumnheadings + mystring)
```

```
a,b,c
1,britt,2.3
```

shows how to create a string from one's data. It is complicated and error prone. It may work for a start or for a very small project, but it does not scale well. In this example I am just printing for display, but if this was being used to save experimental data we would want to use a file handler.

13.0.0.2.1 Opening and Closing a File in Python

```python
a = 1
b = "britt"
c = 2.3
mycolumnheadings = "a," + "b," + "c" + "\n"
mystring = str(a) + "," + b + "," + str(c) + "\n"
with open('/home/britt/gitRepos/uwlooP390/p390/myfile.csv','w+') as f:
    f.write(mycolumnheadings)
    f.write(mystring)
```

Now you can see if you can read that csv into R.

```
mydata = as.data.frame(read.csv("/home/britt/gitRepos/uwlooP390/p390/myfile.csv"))
mydata
```

```
  a     b   c
1 1 britt 2.3
```

13.0.0.2.2 A Better Way: Dictionaries

Python dictionaries (which have their counterparts in most programming languages though they may be called something different) are a convenient way to store data where you can give it a meaningful name. For example, in an experiment with numerous trials and conditions you could have a *key* in your dictionary for "trial no." and another for "condition". The *values* for these would then be updated as your experiment unfolded.

A first pass to using dictionaries is to blend this with a python library for CSV files. Then you can create the column headings from your keys and then each trial that you loop through you can use the dictionary to write a new row to your csv file. Some of the functions you might want to use are:

- `csv.DictWriter`
- `writeheader`
- `writerows`

If you set up and define your dictionary at the start of the file you only need to update the changing values as they occur.

Another approach that will start you getting some familiarity with python pandas library is to make to make a distinct dictionary for each of the trials in your experiment. Then you can concatenate these dictionaries to a list and use that list to create a pandas dataframe. That could then be used in analysis python code or for importing into R. Here is a short example to illustrate the principle.

```
import random as r
import pandas as pd

mylistofkeys=["name","v1","v2","v3","v4"]

mydictlist = []
for i in range(5):
    mydictlist.append({key:value for key,value in zip(["name","v1","v2","v3","v4"],["britt"] + [r

df = pd.DataFrame(mydictlist)

df.loc[:,'v3'].mean()
```

```
np.float64(0.691646472330286)
```

```
myfilename = "mypandastest.csv"
```

```
df.to_csv(myfilename)
```

You can save pandas dataframes as a pickle file, but then you will not be able to easily read it into R. CSV remains the easiest and most direct way to do this. It is probably all you need for most use cases. Though there are other options.

```
rdf = read.csv("mypandastest.csv")
mean(rdf$v3)
```

```
[1] 0.6916465
```

13.0.0.3 Some Analysis Tools

- Pandas TBD
- Some more R TBD

Chapter 14

Knowledge Management

This section deals with the issue of how you keep track of the information you discover and how you mine it for new ideas, associations, and connections. In this section we will consider the notion of the *zettlekasten* and some modern tools for emulating this technique.

For many contemporary adherents to the zettlekasten system their pole star is Niklas Luhman a social systems theorist who was an prolific publisher and who credited much of his fecundity to this technique. Essentially, the method is one of cross-referenced index cards, and historical precedents are easy to find.

Although there are different flavors to this "slip box" approach the consistent theme is that one reflect on their reading, summarize it, and connect it to other works and concepts in their data base. We can perhaps do this more easily [1].

14.1 (Zettlekasten)

1. Org-roam

Org-roam is a tool for use within the Emacs ecosystem. Emacs is, to paraphrase the old joke, an operating system managing as a text editor. Emacs comes with a suite of functions (`org-mode`) built in that facilitate outlining and note taking (and many more things besides). Org-roam is an additional piece of software designed to interact with the Emacs/Orgmode combination to implement many zettlekasten features. It is an emulation, in Emacs, of RoamResearch a self-described tool for networked thought.

The online manual for org-roam provides a brief introduction and overview of the tool. Very, perhaps too, briefly you use various key commands to create nodes

[1] It is a question whether the enhanced digital ease impacts effectiveness since key to the practice is the reading and re-reading of "cards" in the index.

in your network and link them to others. You include notes on entries in your bibliographic system, or free standing notes organized around particular concepts. These links can be viewed both forward (things they link to) or backward (things that link to them), and there are also graphical views. A brief demonstration of some of these tools will be provided in class.

2. Dendron - works with VS Code

This note-taking knowledge management tool is included here because it can interact well with the VS Code and Markdown approach we have been learning about through our use of Quarto.

One of the common features you will find for all these tools is the paring back of dependencies. Most rely on some sort of plain text system at their base. This means you can always just read your entries even if the fancy link and processing features break or prove unwieldy. This generally future proofs your "cards" and also often means that things can work faster and more robustly because there are fewer dependencies.

3. Obsidian

Obsidian is another very popular offering in the same space as the above. This one however will cost you money. I include it here because of its wide popularity. The overview page of the website provides a good general visual orientation to the offerings and features.

Classroom Exercise

Taking Dendron for a test drive.

1. Install Dendron from the VS Code Marketplace.
2. Fill out the survey and any extensions you want (I took all the recommended) and proceed to the tutorial.
3. This should like quite familiar to your. The `qmd` files we have been writing are really just augmented versions of this `md` format.
4. Use the terminal to figure out what directory your notes are being stored in.
5. Use Ctrl-L to create a new note then navigate back to the tutorial.
6. The authors keep talking about `vim`. Look up what `vim` is. Do you have `vim` or its more difficult cousin `vi` on your computer? Try to open one or the other in a terminal.
7. How do you creat a note hierarchy?
8. View the tree mode.
9. Use the Command Palette to view all the Dendron related commands, and at least read the next steps.

14.2 Reference Management

In this section we will look at tools for two important functions related to references for the papers we write and the research we do. How do we find the details for references that we may need to cite, and how do we ease the task of writing so that our references and citations are properly formatted and easy to update when we need to change the citation format for a manuscript.

14.2.1 Finding Citations

- Semantic Scholar

  This tool is supposed to help you find related papers. I have not found it all that helpful in my daily use. Do any of you have success stories to share?

- OpenAlex Try finding my articles here, and see if you can find out what my opinion on *attention* is.

- Google Scholar

  See if you can find any research articles on Albert Einstein's brain by Dr. Sandra Witelson of McMaster University. What I like about this tool is that you can also search for articles that cite particular articles. How many of the articles that cite Dr. Witelson also contain the word myelin?

- Consensus

  This is a new AI Driven search engine. It has some very nice features, but do be careful. Ask it what it thinks about me and category theory. Then look at the references. It is mixing me up with other researchers.

- Undermind

  I think this is a very nice new addition to the search environent, but your free use is very limited. In the paid version you can easily identify articles that seem central to a field of research.

  Sign up and try your free search.

- Research Rabbit To be added.

14.2.2 More on Citation Management

As the web as grown as the principal compendium of scientific articles the community has had to deal with the facts that URLs (website addresses) change. How do we avoid dead links when searching for literature? At the moment, our best tool is to refer to the `doi`.

What does doi stand for?

Find the article with the doi: 10.1371/journal.pone.0152353

There are many tools for this, but a convenient look up is provided by Crossref.

If you keep your own reference database, you will often find that locating the doi is the easiest way to get that reference saved to your database.

The same problem that occurs for uniquely specifiying research articles can also occur for uniquely specifiying researchers. There is more than Britt Anderson out there. How would a reader know if I am the one who wrote a particular article?

What is an orcid?

After you know what one is. Get yourself one if you do **not** already have one.

14.2.3 The last thing you want to do ...

when managing references is to be cut and pasting them into your own home grown document. You want to use a piece of software that can easily keep the data updated and correct, and facilitate you easiliy being able to correctly cite and correctly format your references when you write. Again, there are many tools, but zotero has withstood the test of time and is a free offering well supported by academic libraries world wide.

Zotero is a piece of software, and for this section you ought ot download it and find a few articles on the web, in google scholar, or arxiv that you can install in your database to have a few test articles to work with. The support page will give you links to a quick start guide. There are many ways you can use zotero with tools you have used before (e.g. linking it to a word document), but in a moment we will explore a couple of ways to use it with the Quarto document writing system.

Since we don't want to have to manually format all our citations and references, and then reformat them when we are forced to by needing to resubmit to a new journal we want to rely on the computer to fix such things for us. One contemporary tool that can make that easier is csl. A *csl* file is a file that specifies all the necessary details about how references should be formatted for a particular citation style. Since you will most often be working with apa see if you can find and download the proper csl file https://www.zotero.org/styles, and then at least one other csl type so you can practice how easy it is to switch from one style to another.

14.3 Using References With Quatro and VS Code

Since you will, at least for this course, be writing Qmd files you may want to use Zotero for reference management and connect [it to Quarto](https://quarto.org/docs/visual-editor/technical.html#citations-from-zotero). You will find details under the citation section of this webpage. In addition you will see how doi's, crossref, and other tools can be used to cite while you write in quarto. This workshop also provides details on citation management in Quarto as well as RStudio. See if you can write a minimal stub of an article and try some of the citation managment features in VS Code and Quarto.

14.4 Using References With Overleaf

VS Code is not your only option. Overleaf also use ".bib" files to hold references, though some of the other syntax is different. We have seen this before. Zotero will offer you the option to export references to a bib file which you can then use directly or if you need to submit a `.bib` file for a manuscript.

In addition you can link Overleaf and Zotero to make your writing easier.

Chapter 15

Testing Your Javascript

15.1 Run your own server locally

Running a lab now seems to require some familiarity with servers. And people who want to write their experiments in javascript often want to try things out first so it seems something like XAMMP is a good tool to know about.

15.1.1 XAMPP

XAMMP

15.1.1.1 Downloading

Download link

15.1.2 In Lab Experiments

- Making stuff appear on monitors.
- What is OpenGL?
- Use pygame to make some simple visual experiments.
- Have a beauty contest the following week to see what people have been able to make?
- Make sure that people are using venv

15.1.3 Online Experiments

An exercise with JavaScript? This site has a simple bit of code for throwing an image on the screen.

Can you use the XAMPP server to test it? Require changing the image? Animate a button to toggle or get a random image? WIP HERE

Chapter 16

Large Language Models

16.1 Run locally

Why? Own your own data.

Security and Privacy.

Research applications you may not want participants' data fed to openAI or other proprietary vendors.

16.2 How to?

There are many methods to running LLMs locally on your own hardward. I have chosen GPT4All. It can run under OSX, Windows, and Ubuntu (linux).

16.2.1 Key Features of GPT4ALL

According to the website https://getstream.io/blog/best-local-llm-tools/:

GPT4All can run LLMs on major consumer hardware such as Mac M-Series chips, AMD and NVIDIA GPUs. The following are its key features.

- Privacy First: Keep private and sensitive chat information and prompts only on your machine.
- No Internet Required: It works completely offline.
- Models Exploration: This feature allows developers to browse and download different kinds of LLMs to experiment with. You can select about 1000 open-source language models from popular options like LLama, Mistral, and more.

- Local Documents: You can let your local LLM access your sensitive data with local documents like .pdf and .txt without data leaving your device and without a network.
- Customization options: It provides several chatbot adjustment options like temperature, batch size, context length, etc.
- Enterprise Edition: GPT4ALL provides an enterprise package with security, support, and per-device licenses to bring local AI to businesses.

It is also a very popular LLM for self-hosting (second only to Ollama, which you should also try out).

16.3 How-to

From the GPT4all download page download the version for your operating system.

Follow the download instructions appropriate to your system and pay particular attention to any warnings and make sure that you have sufficient freespace. This is a big application and the model parameters are also quite large.

After you have successfully downloaded and installed the program checkout the quickstart guide.

1. Start Chatting
2. Install a Model
3. I tried “Llama 3.2 1b Instruct”
4. Load the model
5. Start chatting

16.4 Using LLMs for Dynamic Research

What are the mechanics we need for this? We need our model to accept input and generate output and we need a method to transfer the elements of the conversation back and forth. The first stage is to think of the components we have used so far and how we might stitch them together for this task.

It is also a useful transitional step to work incrementally. First, maybe try to set up an interface to allow someone to talk to the model via a web interface. Then you might be able to figure out how to use GET and POST from PHP to communicate between two web servers. There are definitely better ways to do that, but it is a fairly direct starting point. If you want an additional challenge perhaps try to use the tool: curl.

Classroom Exercise

Download and implement the above.
Start the GPT4All Server
See if you can get your GPT4All model to talk to another group's model.

16.5 Why Might You Want To Do This?

Durably reducing conspiracy beliefs through dialogues with AI

https://www.youtube.com/watch?v=qD1fnYDTbHM

This LinkedIn post talks about ChatGPT integration. What would you have to change to get it working with your local server?

Class Question

What is a RAG in this context?

16.6 Next steps

Recently (Duan, Li, and Cai 2024) has published an R package to make all of this easier. The github repository has the library, a "read-me" and installation instructions. Our *class activity* is to see if anyone can get this up and working by next week.

Chapter 17

Databases and management

- What is it
- A book on DuckDB - database stuff
- MariaDB This SQL like, and open source. Might be easier to get started with and still be SQL enough to give them some professional benefits. I was thinking we could get some data online, often they come as CSV's and read it into the database? This is one example how. A blog that compares MariaDB to SQL. A quickie tutorial.
- Why would I want to use a relational database over a csv file (or R data frame or similar)? This could be a class exercise and discussion.
- *Exercise* Download MariaDB

17.0.1 Datascience

- one person's roadmap for non-cs grads

17.0.2 Grant Funding

At the moment I am not sure if will have time for this. But thinking of having the students review the peer review manual for NSERC Discovery Grants. Then have them each write a minimal proposal. Assign the proposals to members of the class, and then hold our own reviewers meeting to decide which projects to fund. Top grants get performed for experiments? Get extra-credit points?

Chapter 18

Summary

In summary, I have not gotten to writing a summary yet.

Appendix A

Mac Related Instructions

A.1 Homebrew

Homebrew installation instructions

A.1.1 Texlive

Get it via Homebrew with `brew install texlive` then export to pdf via quarto works well.

A.2 UV

uv

A.3 Quarto

Find the download here. The demo needs `matplotlib` and `plotly`.

It can be simpler to get python from Homebrew instead of uv with `brew install python-matplotlib` .

Appendix B

Python

Appendix C

Virtual Environments

The advantage of a virtual environment is the ability to isolate libraries you may install for a particular project from interfering with other projects or previously installed libraries. The disadvantage is that they do involve some additional steps and complexity and you may end up installing multiple versions of the same library, which can be an issue if hard disk space on your computer is limited.

C.1 Methods

There is more than one tool for creating a python virutal environment, but one of them is built into the standard python installation: *venv*, and so for simplicity this is the one that I recommend starting with.

C.1.1 Creating and Activating the Venv

If you are in a console you can simply run these commands in the terminal.

```
python -m venv <dir-where-you-want-venv>
cd <dir-where-you-want-venv>
source ./bin/activate
```

> **Note**
>
> Depending on the specifics of your operating system and python installation you may find you need to use `python3` and `pip3` commands instead of plain `python` and `pip`.

If you are using a Windows device you will have to find your equivalent of a linux terminal. This can be using WSL or something like MinGW.

Once you have activated your `venv` you will see a change in the terminal display that indicates the `venv` is active. Now when you run things like `pip install <something>` it will install that library to a sandbox that is only accessible when using python from within that virtual environment. You should note that this notion of a sandbox or virtual environment is also available for some other programming languages.

To deactivate the environment you run ,

```
deactivate
```

from within the venv. Look for the change in the terminal display. Note how activating the environment shows up in the terminal prompt.

```
britt@arts-220486 ~ % source p390venv/bin/activate
(p390venv) britt@arts-220486 ~ % deactivate
britt@arts-220486 ~ %
```

C.1.2 The Terminal is All You Need

If you are a minimalist or doing something simple you can invoke the python interpreter in your venv and complete your task there with no other tools, and using only the libraries required for your particular, limited task. However, if you use some companion tooling things may get more complicated and trickier to debug.

C.2 Visual Studio Code

VS Code tries to make using a venv easier, but this also means there is some indirection that may make it harder to puzzle out errors. The basic work flow is to make sure that VS code is working in the folder you want your venv to be in. If not, use the `open folder` menu of VS Code to do so. Then, you can use the `Command Palette` (found under the *View* menu tab) to locate the `Python Create Environment` command. If you have never created an environment before it will do so. This can take some time so be patient and make sure this process completes. If there was a virtual environment previously constructed VS Code will probably be able to figure that out and ask you if you want to re-use it. Usually you will, but if not you can select to destroy it and create a new one. You will probably also need to affilate a particular python version to association to your environment. VS Code can often make a reasonable suggestion and you can also use the command `Python Select Interpreter` if necessary.

After you have done that you will need to move into the affiliated terminal window of your VS Code environment to install the necessary libraries. You will need to be deliberate and make sure that you are in the correct terminal. VS Code may have several of these open for different purposes if you are working on a complicated project.

Note that while it may be convenient to use the VS Code terminals you do not have to. If you have a terminal program on your computer accessible to you outside of VS Code you can use VS Code as the editor and then move to your other terminal for activating and implementing and compiling the python project.

C.2.1 Quarto Complications

When you embed a python code block in your .qmd document you will want to compile it. Quarto is assembling a document. It will need more than just the libraries necessary for compiling the python code. It also needs the library for the textual and graphical representation of the output that you will insert into your document. Specifically, it needs the `jupyter` python library. This means that whatever the python is doing, even if no additional libraries are needed, you will have to have done a `pip install jupyter` in your venv *and* you will need to have the venv active for the location where the qmd is stored and compiled.

It is worth knowing that although VS Code offers the convenience of buttons to invoke the quarto process it is *not* necessary. You can invoke all the quarto commands through a terminal directly. For instance `quarto preview <filename>.qmd` will try to compile your file and will also usually launch your browser so you can see the html result. It will then watch the source file and update the html whenever it detects you have changed the source. This can be quite convenient for editing.

Duan, Xufeng, Shixuan Li, and Zhenguang G. Cai. 2024. "MacBehaviour: An r Package for Behavioural Experimentation on Large Language Models." *Behavior Research Methods* 57 (1). https://doi.org/10.3758/s13428-024-02524-y.